

Programmation Orientée Objet en Python

Fiche de révision progressive · Exercices sur table (sans ordinateur)

[← Cours P00](#)

★ Automatismes syntaxiques

★★ Logique et comparaisons

★★★ Encapsulation et méthodes

★★★★ Listes d'objets

★★★★★ Niveau Baccalauréat

PARTIE 01 Automatismes syntaxiques ★

RÉFLEXE AVANT TOUT RAISONNEMENT

Exo 1.1 Le rituel du constructeur



On veut modéliser une **Voiture** avec deux attributs : `marque` (chaîne) et `vitesse_max` (entier).

Écris uniquement le constructeur de cette classe. Rien d'autre.

⚠ Règles :

- Le constructeur s'appelle obligatoirement `__init__` (deux underscores de chaque côté).
- Son premier paramètre est toujours `self` — jamais le nom de la classe.
- Chaque attribut se déclare avec `self.nom_attribut = valeur` — le `self.` est obligatoire.

✓ CORRIGÉ COMMENTÉ

```
def __init__(self, marque, vitesse_max):  
    self.marque      = marque      # attribut lié à l'instance  
    self.vitesse_max = vitesse_max # pas de "int vitesse_max" : pas de typage
```

⚡ PIÈGES CLASSIQUES À VERBALISER

Écrire `def init(self, ...)` sans les doubles underscores → Python ignore le constructeur.

Écrire `self = marque` au lieu de `self.marque = marque` → erreur de sémantique totale.

Ajouter un `return` → le constructeur ne renvoie jamais rien (il est *void*).

Exo 1.2 Instanciation correcte



On dispose de la classe suivante :

```
class Livre:
    def __init__(self, titre, pages):
        self.titre = titre
        self.pages = pages
```

Écris la ligne qui crée un objet `mon_livre` représentant "Dune" avec 412 pages.

⚠ **Règle** : On instancie avec `NomClasse(arg1, arg2)` — pas besoin d'écrire `__init__` ni `self` à l'extérieur.

✓ CORRIGÉ COMMENTÉ

```
mon_livre = Livre("Dune", 412)
# Python appelle automatiquement __init__ avec self = mon_livre
```

⚡ PIÈGES CLASSIQUES

Écrire `Livre.__init__("Dune", 412)` → on n'appelle jamais `__init__` manuellement.

Écrire `mon_livre = new Livre(...)` → `new` n'existe pas en Python (confusion avec Java/JS).

Exo 1.3 Chasse au bug — self oublié



Le code ci-dessous contient **3 erreurs de syntaxe POO**. Entoure-les et écris la version corrigée.

```
class Compte:
    def __init__(self, titulaire, solde):
        titulaire = titulaire    # bug 1
        self.solde = solde

    def deposer(montant):        # bug 2
        self.solde = self.solde + montant

    def afficher(self):
        print(self.solde AND self.titulaire) # bug 3
```

⚠ **Règles** : Toute méthode de classe prend `self` en premier paramètre. Les opérateurs logiques Python sont en **minuscules** : `and`, `or`, `not`.

✓ CORRIGÉ COMMENTÉ

```
class Compte:
    def __init__(self, titulaire, solde):
        self.titulaire = titulaire # bug 1 : self. manquant
        self.solde = solde

    def deposter(self, montant): # bug 2 : self manquant en 1er paramètre
        self.solde = self.solde + montant

    def afficher(self):
        print(self.solde, self.titulaire) # bug 3 : AND → virgule ou autre la
```

⚡ PIÈGES CLASSIQUES

Bug 1 : sans `self.`, on crée une variable locale qui disparaît à la fin de `__init__`.

Bug 2 : Python passera l'objet en argument implicite → décalage de tous les paramètres → `TypeError`.

Bug 3 : `AND` en majuscules est une *variable non définie* en Python, pas un opérateur.

Exo 1.4 Attribut vs Variable locale



On crée un objet `p = Point(3, 5)`. **Complète les pointillés** pour accéder à son abscisse :

a) Pour lire l'abscisse depuis l'extérieur de la classe : valeur = `p. ....`

b) Pour lire l'abscisse *depuis une méthode* de la classe : valeur = `.....`.x

⚠ Règle : Un attribut n'est jamais appelé avec des parenthèses — ce n'est pas une méthode.

✓ CORRIGÉ COMMENTÉ

```
# a) depuis l'extérieur
valeur = p.x # nom_objet.nom_attribut (pas de parenthèses !)

# b) depuis une méthode
valeur = self.x # self remplace le nom de l'objet à l'intérieur de la cla
```

⚡ PIÈGES CLASSIQUES

Écrire `p.x()` → Python cherche une *méthode* appelée `x` → `TypeError: 'int' object is not callable`.
Écrire `x` seul dans une méthode (sans `self.`) → Python cherche une variable globale, pas l'attribut.

PARTIE 02 Logique et comparaisons ★★

MÉTHODES AVEC CONDITIONS

Exo 2.1 Méthode de validation – Étudiant ★★

La classe `Etudiant` possède deux attributs : `note` (float, 0-20) et `absences` (int).

Écris la méthode `est_admis(self)` qui retourne `True` si la note est **supérieure ou égale à 10** et le nombre d'absences est **strictement inférieur à 5**, `False` sinon.

⚠ Règles :

- Opérateurs de comparaison : `>=` et `<` (pas de `=>` ou `=<`).
- Opérateur logique : `and` en minuscules (jamais `AND` ou `&&`).
- Cette méthode **doit** utiliser un `return`.

✓ CORRIGÉ COMMENTÉ

```
def est_admis(self):  
    return self.note >= 10 and self.absences < 5  
    # On peut aussi écrire :  
    if self.note >= 10 and self.absences < 5:  
        return True  
    else:  
        return False
```

⚡ PIÈGES CLASSIQUES

`AND` majuscule → `NameError`. `&&` → `SyntaxError`.

Écrire `10 <= self.note` est valide mais moins lisible — à déconseiller en copie.

Oublier `self.` devant `note` ou `absences` → `NameError`.

Exo 2.2 Méthode de catégorie – Produit ★★

La classe `Produit` a un attribut `prix` (float). Écris la méthode `categorie(self)` qui retourne :

- "économique" si le prix est strictement inférieur à 20

- "standard" si le prix est entre 20 et 100 inclus
- "premium" sinon

⚠ Règles :

- Utilise des `if` / `elif` / `else` imbriqués, pas des `if` successifs.
- Une seule sortie de méthode : chaque branche contient un `return`.

✓ CORRIGÉ COMMENTÉ

```
def categorie(self):  
    if self.prix < 20:  
        return "économique"  
    elif self.prix ≤ 100:    # implicitement prix ≥ 20 ici  
        return "standard"  
    else:  
        return "premium"
```

⚡ PIÈGES CLASSIQUES

Utiliser `if ... if ... if` au lieu de `elif` → tous les blocs s'évaluent indépendamment, plusieurs `return` possibles.

Oublier que `elif self.prix ≤ 100` hérite déjà de la condition précédente `prix ≥ 20`.

Retourner un entier 0/1/2 au lieu d'une chaîne → la méthode ne répond pas au contrat demandé.

Exo 2.3 Méthode de modification – Température

★★

La classe `Thermostat` a un attribut `temperature` (float). Écris la méthode `chauffer(self, delta)` qui **augmente** la température de `delta` degrés, mais **uniquement si** le résultat ne dépasse pas 30°C.

⚠ Règles :

- Une méthode de *modification* ne renvoie rien (**pas de return**).
- On modifie l'attribut avec `self.temperature = ...`

✓ CORRIGÉ COMMENTÉ

```
def chauffer(self, delta):  
    if self.temperature + delta ≤ 30:  
        self.temperature = self.temperature + delta  
    # Pas de return : la méthode modifie l'état, elle ne renvoie rien
```

⚡ PIÈGES CLASSIQUES

Écrire `return self.temperature` → on ne renvoie pas l'attribut modifié dans une méthode mutateur.
Écrire `temperature = self.temperature + delta` (sans `self.`) → variable locale perdue, l'attribut reste inchangé.
Tester `if delta ≤ 30` au lieu de `if self.temperature + delta ≤ 30` → la condition est incorrecte.

PARTIE
03

Encapsulation et méthodes



LIRE ET ÉCRIRE LES ATTRIBUTS
PROPREMENT

Exo 3.1 Getter et Setter – Batterie



La classe `Batterie` possède un attribut `charge` (int, entre 0 et 100). Écris :

- La méthode `get_charge(self)` qui retourne la charge actuelle.
- La méthode `set_charge(self, nouvelle_charge)` qui modifie la charge **seulement si** `nouvelle_charge` est entre 0 et 100 inclus.

⚠ Règles :

- Le getter utilise `return` — le setter **n'en utilise pas**.
- On teste avec `0 ≤ nouvelle_charge ≤ 100` (enchaînement de comparaisons valide en Python).

✓ CORRIGÉ COMMENTÉ

```
def get_charge(self):  
    return self.charge          # accesseur : retourne l'attribut  
  
def set_charge(self, nouvelle_charge):  
    if 0 ≤ nouvelle_charge ≤ 100:  
        self.charge = nouvelle_charge # mutateur : modifie sans return
```

⚡ PIÈGES CLASSIQUES

Confondre `get_charge()` et `self.charge` à l'appel : appeler la méthode nécessite des parenthèses `b.get_charge()`, pas `b.get_charge`.

Écrire `nouvelle_charge ≥ 0 AND nouvelle_charge ≤ 100` → AND invalide + redondant avec la syntaxe Python.

Exo 3.2 Méthode de comparaison – Rectangle



La classe Rectangle possède deux attributs : largeur et hauteur. Écris :

- La méthode `aire(self)` qui retourne l'aire.
- La méthode `est_plus_grand(self, autre)` qui retourne `True` si **l'aire de self est strictement supérieure** à l'aire de `autre`.

⚠ **Règle** : Dans `est_plus_grand`, appelle la méthode `aire()` avec des parenthèses — ce n'est pas un attribut.

✓ CORRIGÉ COMMENTÉ

```
def aire(self):  
    return self.largeur * self.hauteur  
  
def est_plus_grand(self, autre):  
    return self.aire() > autre.aire()  
    # aire() → méthode → parenthèses OBLIGATOIRES  
    # autre est aussi un objet Rectangle : on lui applique la même méthode
```

⚡ PIÈGES CLASSIQUES

Écrire `self.aire > autre.aire` (sans parenthèses) → on compare *des références de méthodes*, pas des entiers → résultat toujours `True` car les objets "méthode" sont distincts.

Recalculer manuellement `self.largeur * self.hauteur` dans `est_plus_grand` → code dupliqué, jamais réutiliser ce qu'une méthode fait déjà.

Exo 3.3 Méthode `__str__` — Affichage d'objet

★★★

La classe `Eleve` a les attributs `nom` et `note`. Écris la méthode spéciale `__str__(self)` qui retourne une chaîne du type "Alice : 14/20".

⚠ **Règle** : `__str__` doit toujours return une chaîne — jamais utiliser `print` à l'intérieur.

✓ CORRIGÉ COMMENTÉ

```
def __str__(self):  
    return self.nom + " : " + str(self.note) + "/20"  
    # Ou avec f-string (Python 3.6+) :  
    return f"{self.nom} : {self.note}/20"
```

⚡ PIÈGES CLASSIQUES

Écrire `print(self.nom + ...)` dans `__str__` → `print` retourne `None`, pas une chaîne.
Concaténer sans `str(self.note)` → `TypeError: can only concatenate str (not "int") to str`.

PARTIE
04

Manipulation de listes d'objets



BOUCLES SUR DES TABLEAUX
D'INSTANCES

Exo 4.1 Cumul d'attributs – Bibliothèque



On dispose de la classe `Livre` avec les attributs `titre` (`str`) et `pages` (`int`). On a une liste `bibliotheque` contenant plusieurs objets `Livre`.

Écris la **fonction** `total_pages(bibliotheque)` qui retourne le nombre total de pages de tous les livres.

⚠ Règles :

- C'est une *fonction* (hors classe), pas une méthode.
- On accède à l'attribut `pages` sans parenthèses : `livre.pages`, pas `livre.pages()`.
- Initialiser le compteur à 0 avant la boucle.

✓ CORRIGÉ COMMENTÉ

```
def total_pages(bibliotheque):  
    total = 0                # accumulateur initialisé à 0  
    for livre in bibliotheque:  
        total = total + livre.pages    # livre est un objet Livre  
    return total             # attribut → pas de parenthèses !
```

⚡ PIÈGES CLASSIQUES

Écrire `livre.pages()` → `TypeError: 'int' object is not callable` — `pages` est un attribut, pas une méthode.

Oublier d'initialiser `total = 0` → `NameError` ou résultat faux.

Écrire `for i in range(len(bibliotheque))` puis `bibliotheque[i].pages` → fonctionne, mais boucle `for x in liste` est plus pythonique et attendue au bac.

Exo 4.2 Appel de méthode en boucle – Promotion



La classe `Eleve` possède l'attribut `note` et la méthode `est_admis()` (sans paramètre autre que `self`, retourne un booléen).

Écris la fonction `compter_admis(promo)` qui prend une liste d'objets `Eleve` et retourne le nombre d'élèves admis.

⚠ Règles :

- Appeler une méthode nécessite des parenthèses : `eleve.est_admis()`.
- Ne pas réécrire la condition interne de `est_admis` — la méthode existe, utilise-la.

✓ CORRIGÉ COMMENTÉ

```
def compter_admis(promo):
    compteur = 0
    for eleve in promo:
        if eleve.est_admis():          # méthode → parenthèses obligatoires
            compteur = compteur + 1
    return compteur
```

⚡ PIÈGES CLASSIQUES

Écrire `if eleve.est_admis` (sans parenthèses) → la condition est *toujours* vraie : un objet méthode est `truthy`.

Réécrire `if eleve.note ≥ 10` au lieu d'appeler `est_admis()` → duplication de logique et non-respect de l'encapsulation.

Exo 4.3 Recherche du maximum — Tournoi

★★★★

La classe `Joueur` possède les attributs `nom` (`str`) et `score` (`int`). Écris la fonction `meilleur_joueur(joueurs)` qui retourne l'**objet** `Joueur` ayant le score le plus élevé. On suppose que la liste est non vide.

⚠ Règles :

- Initialise ton meilleur candidat avec le premier élément de la liste : `meilleur = joueurs[0]`.
- Retourne l'*objet entier*, pas uniquement son score.

✓ CORRIGÉ COMMENTÉ

```
def meilleur_joueur(joueurs):
    meilleur = joueurs[0]          # objet Joueur initial
    for j in joueurs:
        if j.score > meilleur.score:  # compare les attributs
            meilleur = j             # on mémorise l'objet, pas juste le score
    return meilleur                # retourne l'objet complet
```

⚡ PIÈGES CLASSIQUES

Initialiser meilleur = 0 → on compare un entier à un objet → AttributeError.
Retourner meilleur.score au lieu de meilleur → on perd le nom et l'objet complet.
Écrire meilleur = None et oublier de gérer le cas initial → AttributeError: 'NoneType' has no attribute 'score'.

PARTIE
05

Niveau Baccalauréat



CLASSE COMPLÈTE • CALCULS COMPLEXES •
CONTEXTE RÉEL

Exo 5.1 Classe Cinéma – Moyenne pondérée



On modélise un film avec la classe suivante (partiellement écrite) :

```
class Film:
    def __init__(self, titre, duree, note_presse, note_public):
        self.titre = titre
        self.duree = duree # en minutes
        self.note_presse = note_presse # sur 10
        self.note_public = note_public # sur 10
```

Question 1. Ajoute la méthode note_globale(self) qui retourne la moyenne pondérée : 40% presse + 60% public.

Question 2. Écris la fonction films_premium(catalogue) qui prend une liste d'objets Film et retourne la **liste** de tous les films dont la note globale est supérieure ou égale à 7.

⚠ Règles :

- Dans films_premium, appelle note_globale() avec ses parenthèses.
- La fonction retourne une *liste* d'objets Film, pas leurs titres.
- Construis la liste résultat avec resultat = [] puis resultat.append(...).

✓ CORRIGÉ COMMENTÉ

```
# Question 1 – méthode de la classe Film
def note_globale(self):
    return 0.4 * self.note_presse + 0.6 * self.note_public

# Question 2 – fonction hors classe
def films_premium(catalogue):
    resultat = []
    for film in catalogue:
        if film.note_globale() ≥ 7: # méthode → parenthèses
```

```
        resultat.append(film)           # on ajoute l'objet Film entier
    return resultat
```

⚡ PIÈGES CLASSIQUES

Écrire `film.note_globale ≥ 7` → compare une référence de méthode à un entier → `TypeError`.
Appender `film.titre` → on stocke des chaînes, pas des objets → l'appelant ne peut plus accéder aux autres attributs.
Mélanger 40% et 60% → lire l'énoncé : presse est minoritaire dans la pondération ici.

Exo 5.2 Classe Banque – Méthode de classe + traitement de liste

★★★★★

On modélise un compte bancaire :

```
class Compte:
    def __init__(self, titulaire, solde):
        self.titulaire = titulaire
        self.solde      = solde

    def est_debiteur(self):
        return self.solde < 0

    def prelever(self, montant):
        if montant > 0 and self.solde ≥ montant:
            self.solde = self.solde - montant
```

Question 1. Écris la fonction `solde_total(comptes)` qui retourne la somme de tous les soldes de la liste (comptes positifs et négatifs).

Question 2. Écris la fonction `titulaires_debiteurs(comptes)` qui retourne la *liste des noms* (attribut `titulaire`) des comptes débiteurs.

⚠ Règles :

- Dans Q2, utilise la méthode `est_debiteur()` — ne réécris pas la condition.
- Q2 retourne une liste de *chaînes* (noms), pas d'objets `Compte`.

✓ CORRIGÉ COMMENTÉ

```
# Question 1
def solde_total(comptes):
    total = 0
    for c in comptes:
        total = total + c.solde    # attribut : pas de parenthèses
    return total
```

Question 2

```
def titulaires_debiteurs(comptes):
    debiteurs = []
    for c in comptes:
        if c.est_debiteur():          # méthode → parenthèses
            debiteurs.append(c.titulaire) # attribut : pas de parenthèses
    return debiteurs
```

⚡ PIÈGES CLASSIQUES

Dans Q1: `c.solde()` → `TypeError` — `solde` est un attribut.

Dans Q2: `c.est_debiteur` sans `()` → comparaison d'une référence de méthode, toujours `True`.

Dans Q2: `append c` (l'objet) au lieu de `c.titulaire` (le nom) → la liste contiendrait des objets `Compte`, pas des chaînes.

Exo 5.3 Classe Météo – Classe complète + analyse

★★★★★

On veut modéliser des relevés météorologiques. Chaque relevé a : un jour (str), une température (float) et des précipitations (float, en mm).

Question 1. Écris la classe `Releve` complète avec son constructeur et la méthode `est_pluvieux(self)` qui retourne `True` si les précipitations dépassent strictement 5 mm.

Question 2. Écris la fonction `temperature_moyenne(relevés)` qui calcule et retourne la moyenne des températures. Si la liste est vide, retourner 0.

Question 3. Écris la fonction `jours_pluvieux(relevés)` qui retourne la liste des *jours* (attribut `jour`) où il a plu.

⚠ Règles :

- Q2 : protège contre la division par zéro avec `if len(relevés) == 0`.
- Q3 : appelle `est_pluvieux()` avec ses parenthèses.

✓ CORRIGÉ COMMENTÉ

Question 1 – Classe complète

```
class Releve:
    def __init__(self, jour, temperature, precipitations):
        self.jour = jour
        self.temperature = temperature
        self.precipitations = precipitations

    def est_pluvieux(self):
        return self.precipitations > 5
```

Question 2

```
def temperature_moyenne(relevés):
```

```
if len(relevés) == 0:
    return 0
total = 0
for r in relevés:
    total = total + r.temperature
return total / len(relevés)
```

Question 3

```
def jours_pluvieux(relevés):
    jours = []
    for r in relevés:
        if r.est_pluvieux():      # méthode → parenthèses
            jours.append(r.jour)  # attribut → pas de parenthèses
    return jours
```

⚡ PIÈGES CLASSIQUES

Q1 : oublier l'un des trois `self.attribut = parametre` → l'attribut sera inaccessible ensuite → `AttributeError`.

Q2 : diviser sans vérifier la longueur → `ZeroDivisionError` si liste vide.

Q3 : appender `r` (l'objet entier) au lieu de `r.jour` (la chaîne) → mélange des types dans la liste retournée.